



US 20250278669A1

(19) **United States**

(12) **Patent Application Publication**
Martinez Gil et al.

(10) **Pub. No.: US 2025/0278669 A1**

(43) **Pub. Date: Sep. 4, 2025**

(54) **COUNTERFACTUALS WITH FEATURE
PREFERENCES FOR CONSISTENT AND
DIVERSE EXPLANATIONS**

(71) Applicant: **International Business Machines
Corporation**, Armonk, NY (US)

(72) Inventors: **Natalia Lucienne Martinez Gil**,
Ossining, NY (US); **Kanthi Sarpatwar**,
Briarcliff Manor, NY (US); **Sumanta
Mukherjee**, Bangalore (IN); **Amaresh
Rajasekharan**, Wappingers Falls, NY
(US); **Roman Vaculin**, Larchmont, NY
(US)

(73) Assignee: **International Business Machines
Corporation**, Armonk, NY (US)

(21) Appl. No.: **18/594,202**

(22) Filed: **Mar. 4, 2024**

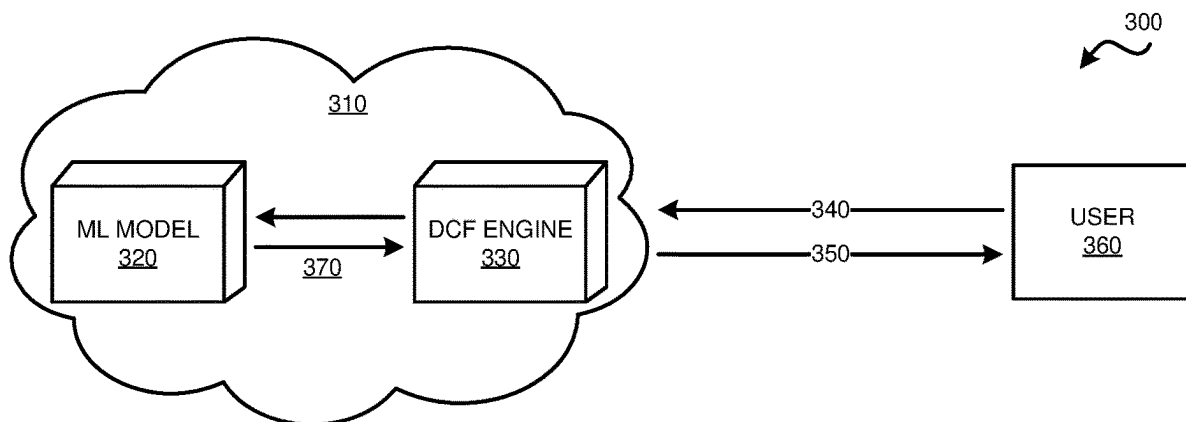
Publication Classification

(51) **Int. Cl.**
G06N 20/00 (2019.01)

(52) **U.S. Cl.**
CPC **G06N 20/00** (2019.01)

(57) **ABSTRACT**

An embodiment includes detecting by an artificial intelligence system a feature of a machine learning model. The embodiment includes responsive to detecting the feature of the machine learning model, computing by a Counterfactual Engine of the artificial intelligence system a counterfactual objective based on modifying a product of a weight and a perturbation of the feature. The embodiment also includes transforming by the Counterfactual Engine a counterfactual based on the counterfactual objective.



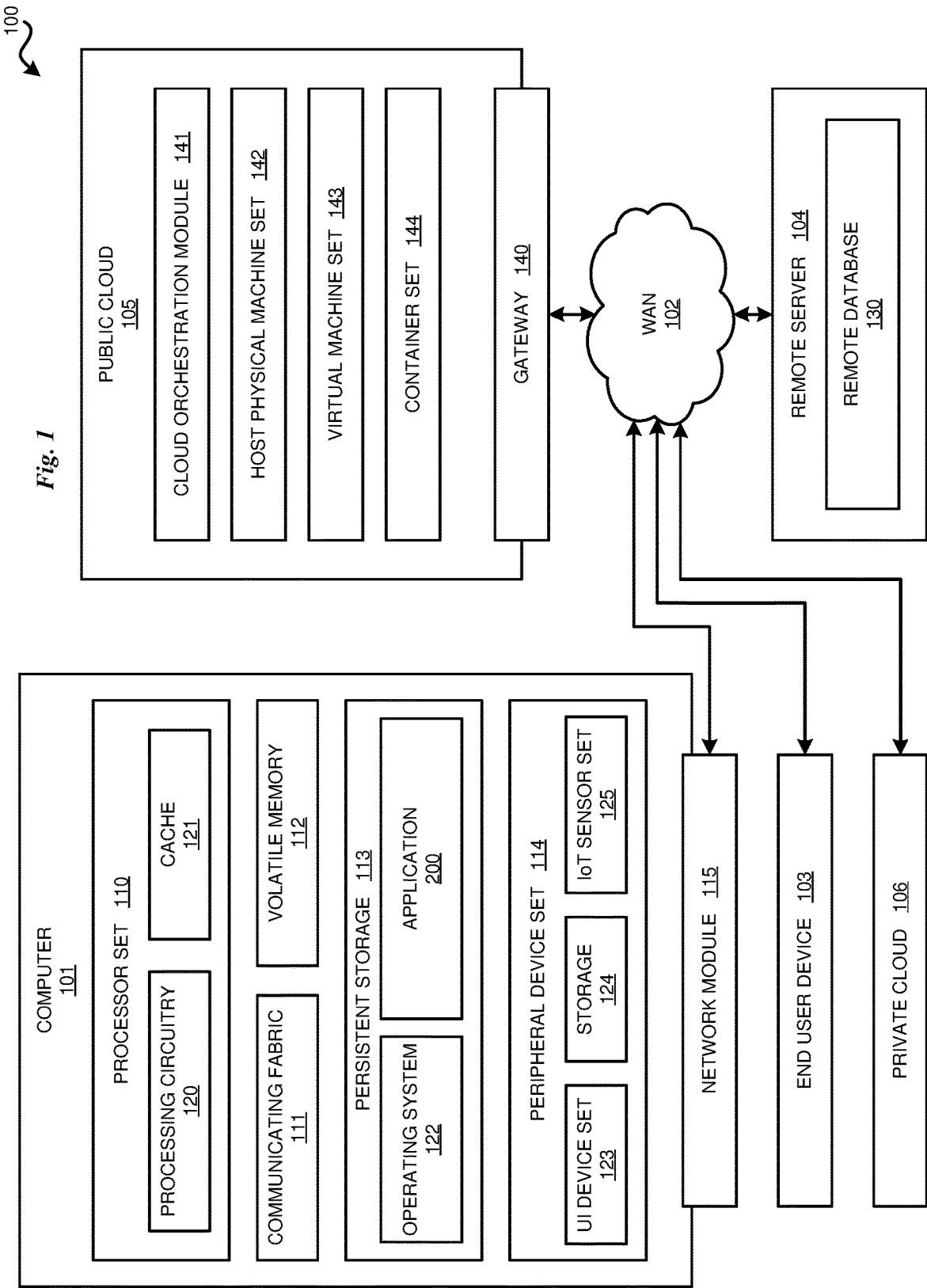


Fig. 2

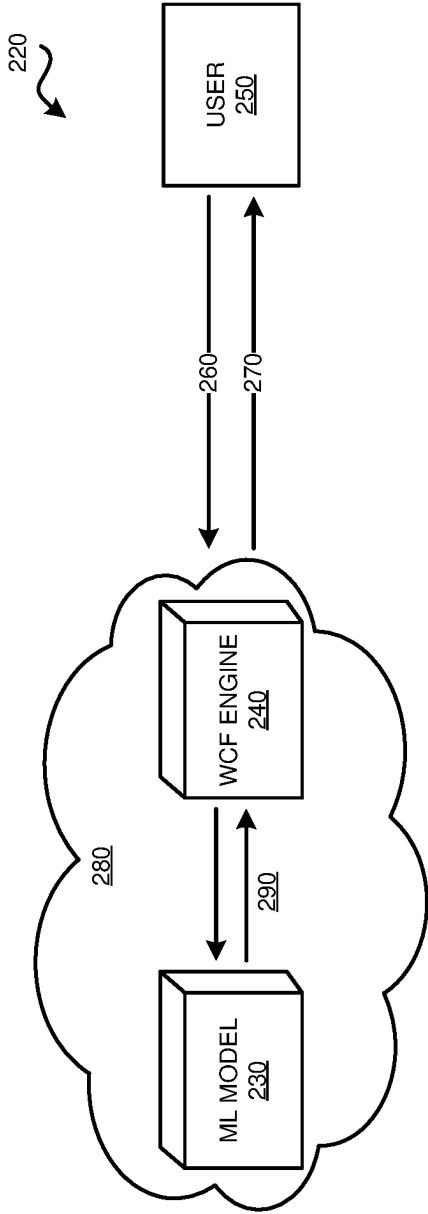


Fig. 3

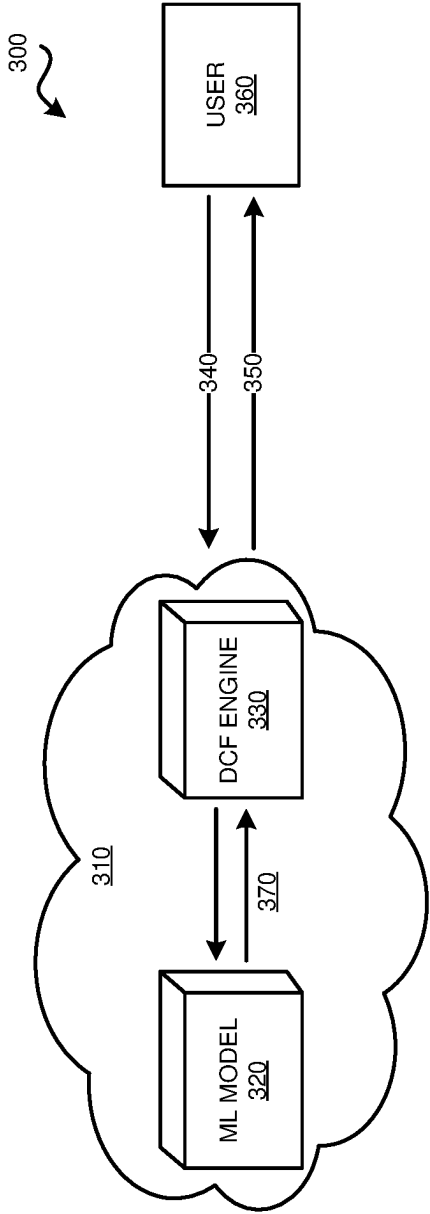


Fig. 4

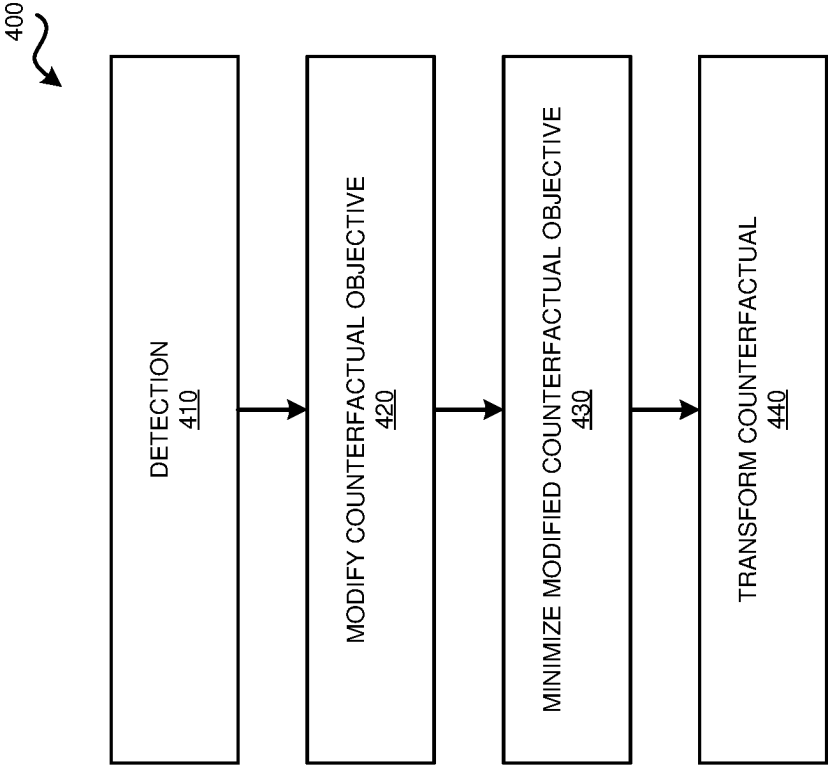


Fig. 5

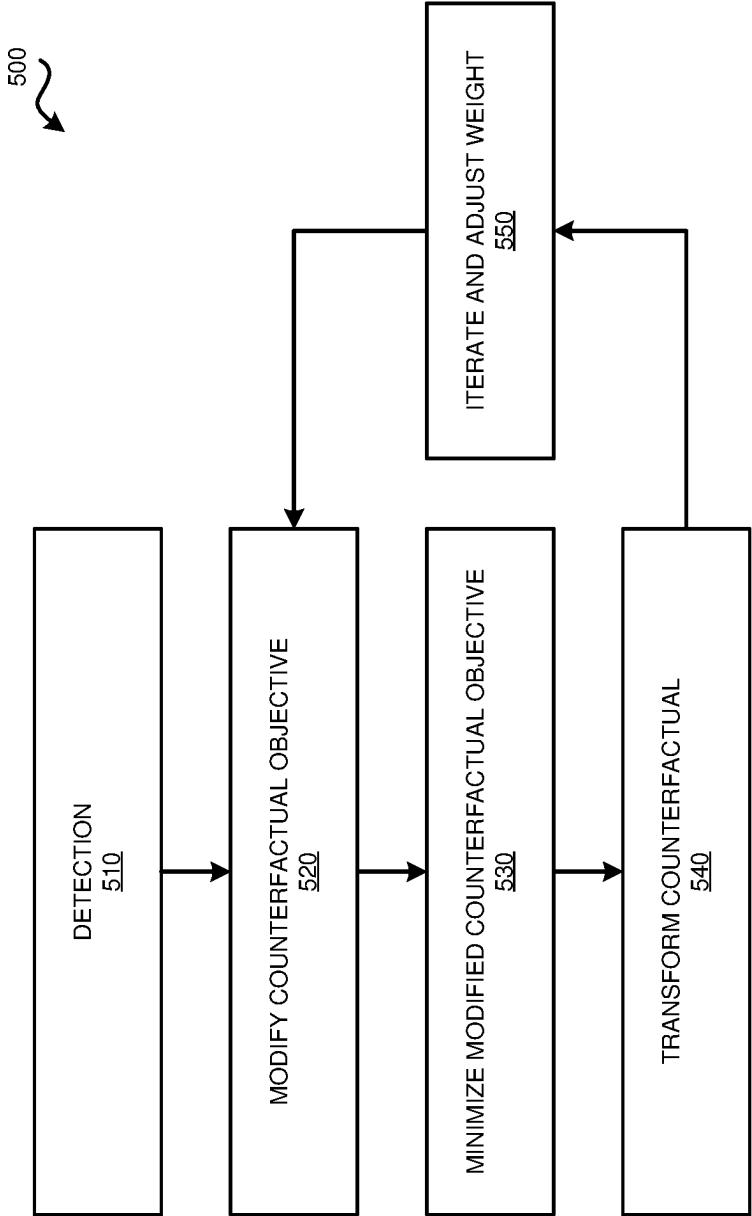
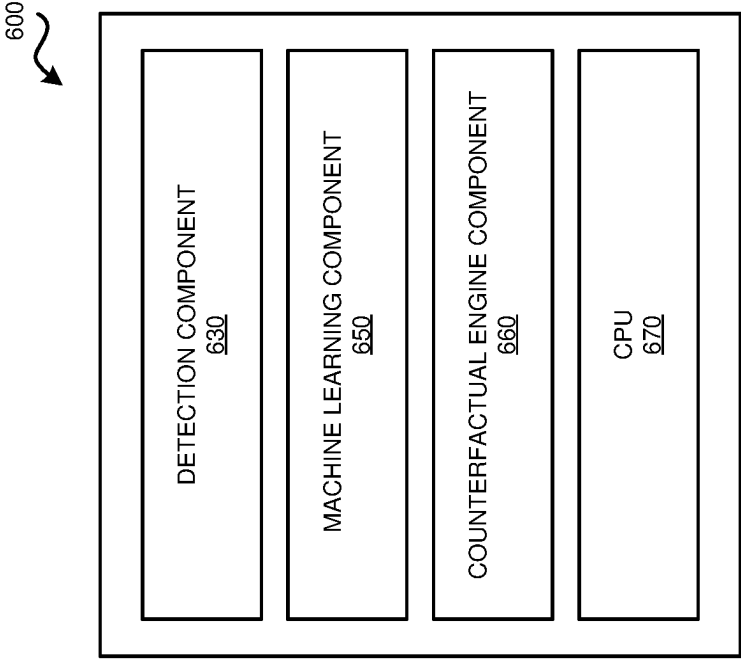


Fig. 6



COUNTERFACTUALS WITH FEATURE PREFERENCES FOR CONSISTENT AND DIVERSE EXPLANATIONS

BACKGROUND

[0001] The present invention relates generally to artificial intelligence. More particularly, the present invention relates to a method, system, and computer program for Counterfactuals with Feature Preferences for Consistent and Diverse Explanations.

[0002] Complex machine learning models such as deep neural networks (DNNs) or large ensembles of models have become increasingly popular due to their state-of-the-art performance across multiple prediction tasks. Explainability in Artificial Intelligence (AI) has recently gained popularity. Counterfactual explanations provide actionable insights into the minimal change in a system that would lead to a more desirable prediction from a black box model. Counterfactuals attempt to explain a model outcome in terms of the minimal perturbation needed to change the model outcome (potentially to a more desirable one) for a given instance or input.

SUMMARY

[0003] The illustrative embodiments provide for Counterfactuals with Feature Preferences for Consistent and Diverse Explanations. An embodiment includes detecting by an artificial intelligence system a feature of a machine learning model. The embodiment includes responsive to detecting the feature of the machine learning model, computing by a Counterfactual Engine of the artificial intelligence system a counterfactual objective based on modifying a product of a weight and a perturbation of the feature. The embodiment also includes transforming by the Counterfactual Engine a counterfactual based on the counterfactual objective.

[0004] An embodiment includes a computer usable program product. The computer usable program product includes a computer-readable storage medium, and program instructions stored on the storage medium.

[0005] An embodiment includes a computer system. The computer system includes a processor, a computer-readable memory, and a computer-readable storage medium, and program instructions stored on the storage medium for execution by the processor via the memory.

BRIEF DESCRIPTION OF THE DRAWINGS

[0006] The novel features believed characteristic of the invention are set forth in the appended claims. The invention itself, however, as well as a preferred mode of use, further objectives, and advantages thereof, will best be understood by reference to the following detailed description of the illustrative embodiments when read in conjunction with the accompanying drawings, wherein:

[0007] FIG. 1 depicts a block diagram of a computing environment in accordance with an illustrative embodiment;

[0008] FIG. 2 depicts a diagram of a weighted counterfactual explanation in an environment in accordance with an illustrative embodiment;

[0009] FIG. 3 depicts a diagram of a diverse counterfactual explanation in accordance with an illustrative embodiment;

[0010] FIG. 4 depicts flowchart diagram of a weighted counterfactual explanation in accordance with an illustrative embodiment;

[0011] FIG. 5 depicts flowchart diagram of a diverse counterfactual explanation in accordance with an illustrative embodiment; and

[0012] FIG. 6 depicts a system diagram in accordance with an illustrative embodiment.

DETAILED DESCRIPTION

[0013] Complex machine learning models such as deep neural networks (DNNs) or large ensembles of models have become increasingly popular due to their state-of-the-art performance across multiple prediction tasks. Explainability in Artificial Intelligence (AI) has recently gained popularity. Counterfactual explanations provide actionable insights into the minimal change in a system that would lead to a more desirable prediction from a black box model. Counterfactuals attempt to explain a model outcome in terms of the minimal perturbation needed to change the model outcome (potentially to a more desirable one) for a given instance or input.

[0014] The present disclosure provides a method, a machine-readable medium, and a system for Counterfactuals with Feature Preferences for Consistent and Diverse Explanations. An embodiment includes detecting by an artificial intelligence system a feature of a machine learning model. The embodiment includes responsive to detecting the feature of the machine learning model, computing by a Counterfactual Engine of the artificial intelligence system a counterfactual objective based on modifying a product of a weight and a perturbation of the feature. The embodiment also includes transforming by the Counterfactual Engine a counterfactual based on the counterfactual objective.

[0015] Illustrative embodiments include wherein the weight penalizes the perturbation of the feature.

[0016] Illustrative embodiments include wherein the weight is a maximum value for the feature that is invariant.

[0017] Illustrative embodiments include wherein the modifying further comprises minimizing the product based on a distance algorithm.

[0018] Illustrative embodiments include wherein the weight comprises a cost preference.

[0019] Illustrative embodiments include further comprising iterating the computing and the transforming wherein the iterating updates a set of counterfactuals and a current weighting vector.

[0020] Illustrative embodiments also include wherein the current weighting vector is computed based on a previous counterfactual from the set of counterfactuals.

[0021] For the sake of clarity of the description, and without implying any limitation thereto, the illustrative embodiments are described using some example configurations. From this disclosure, those of ordinary skill in the art will be able to conceive many alterations, adaptations, and modifications of a described configuration for achieving a described purpose, and the same are contemplated within the scope of the illustrative embodiments.

[0022] Furthermore, simplified diagrams of the data processing environments are used in the figures and the illustrative embodiments. In an actual computing environment, additional structures or components that are not shown or described herein, or structures or components different from

those shown but for a similar function as described herein may be present without departing the scope of the illustrative embodiments.

[0023] Furthermore, the illustrative embodiments are described with respect to specific actual or hypothetical components only as examples. Any specific manifestations of these and other similar artifacts are not intended to be limiting to the invention. Any suitable manifestation of these and other similar artifacts can be selected within the scope of the illustrative embodiments.

[0024] The examples in this disclosure are used only for the clarity of the description and are not limiting to the illustrative embodiments. Any advantages listed herein are only examples and are not intended to be limiting to the illustrative embodiments. Additional or different advantages may be realized by specific illustrative embodiments. Furthermore, a particular illustrative embodiment may have some, all, or none of the advantages listed above.

[0025] Furthermore, the illustrative embodiments may be implemented with respect to any type of data, data source, or access to a data source over a data network. Any type of data storage device may provide the data to an embodiment of the invention, either locally at a data processing system or over a data network, within the scope of the invention. Where an embodiment is described using a mobile device, any type of data storage device suitable for use with the mobile device may provide the data to such embodiment, either locally at the mobile device or over a data network, within the scope of the illustrative embodiments.

[0026] The illustrative embodiments are described using specific code, computer readable storage media, high-level features, designs, architectures, protocols, layouts, schematics, and tools only as examples and are not limiting to the illustrative embodiments. Furthermore, the illustrative embodiments are described in some instances using particular software, tools, and data processing environments only as an example for the clarity of the description. The illustrative embodiments may be used in conjunction with other comparable or similarly purposed structures, systems, applications, or architectures. For example, other comparable mobile devices, structures, systems, applications, or architectures therefor, may be used in conjunction with such embodiment of the invention within the scope of the invention. An illustrative embodiment may be implemented in hardware, software, or a combination thereof.

[0027] The examples in this disclosure are used only for the clarity of the description and are not limiting to the illustrative embodiments. Additional data, operations, actions, tasks, activities, and manipulations will be conceivable from this disclosure and the same are contemplated within the scope of the illustrative embodiments.

[0028] Various aspects of the present disclosure are described by narrative text, flowcharts, block diagrams of computer systems and/or block diagrams of the machine logic included in computer program product (CPP) embodiments. With respect to any flowcharts, depending upon the technology involved, the operations can be performed in a different order than what is shown in a given flowchart. For example, again depending upon the technology involved, two operations shown in successive flowchart blocks may be performed in reverse order, as a single integrated step, concurrently, or in a manner at least partially overlapping in time.

[0029] A computer program product embodiment (“CPP embodiment” or “CPP”) is a term used in the present disclosure to describe any set of one, or more, storage media (also called “mediums”) collectively included in a set of one, or more, storage devices that collectively include machine readable code corresponding to instructions and/or data for performing computer operations specified in a given CPP claim. A “storage device” is any tangible device that can retain and store instructions for use by a computer processor. Without limitation, the computer readable storage medium may be an electronic storage medium, a magnetic storage medium, an optical storage medium, an electromagnetic storage medium, a semiconductor storage medium, a mechanical storage medium, or any suitable combination of the foregoing. Some known types of storage devices that include these mediums include: diskette, hard disk, random access memory (RAM), read-only memory (ROM), erasable programmable read-only memory (EPROM or Flash memory), static random-access memory (SRAM), compact disc read-only memory (CD-ROM), digital versatile disk (DVD), memory stick, floppy disk, mechanically encoded device (such as punch cards or pits/lands formed in a major surface of a disc) or any suitable combination of the foregoing. A computer readable storage medium, as that term is used in the present disclosure, is not to be construed as storage in the form of transitory signals per se, such as radio waves or other freely propagating electromagnetic waves, electromagnetic waves propagating through a waveguide, light pulses passing through a fiber optic cable, electrical signals communicated through a wire, and/or other transmission media. As will be understood by those of skill in the art, data is typically moved at some occasional points in time during normal operations of a storage device, such as during access, de-fragmentation or garbage collection, but this does not render the storage device as transitory because the data is not transitory while it is stored.

[0030] With reference to FIG. 1, this figure depicts a block diagram of a computing environment 100. Data center environment 100 contains an example of an environment for the execution of at least some of the computer code involved in performing the inventive methods, such as an Application module 200 that provides Counterfactuals with Feature Preferences for Consistent and Diverse Explanations. In addition to block 200, computing environment 100 includes, for example, computer 101, wide area network (WAN) 102, end user device (EUD) 103, remote server 104, public cloud 105, and private cloud 106. In this embodiment, computer 101 includes processor set 110 (including processing circuitry 120 and cache 121), communication fabric 111, volatile memory 112, persistent storage 113 (including operating system 122 and block 200, as identified above), peripheral device set 114 (including user interface (UI) device set 123, storage 124, and Internet of Things (IoT) sensor set 125), and network module 115. Remote server 104 includes remote database 130. Public cloud 105 includes gateway 140, cloud orchestration module 141, host physical machine set 142, virtual machine set 143, and container set 144.

[0031] COMPUTER 101 may take the form of a desktop computer, laptop computer, tablet computer, smart phone, smart watch or other wearable computer, mainframe computer, quantum computer or any other form of computer or mobile device now known or to be developed in the future that is capable of running a program, accessing a network or

querying a database, such as remote database **130**. As is well understood in the art of computer technology, and depending upon the technology, performance of a computer-implemented method may be distributed among multiple computers and/or between multiple locations. On the other hand, in this presentation of computing environment **100**, detailed discussion is focused on a single computer, specifically computer **101**, to keep the presentation as simple as possible. Computer **101** may be located in a cloud, even though it is not shown in a cloud in FIG. 1. On the other hand, computer **101** is not required to be in a cloud except to any extent as may be affirmatively indicated.

[0032] PROCESSOR SET **110** includes one, or more, computer processors of any type now known or to be developed in the future. Processing circuitry **120** may be distributed over multiple packages, for example, multiple, coordinated integrated circuit chips. Processing circuitry **120** may implement multiple processor threads and/or multiple processor cores. Cache **121** is memory that is located in the processor chip package(s) and is typically used for data or code that should be available for rapid access by the threads or cores running on processor set **110**. Cache memories are typically organized into multiple levels depending upon relative proximity to the processing circuitry. Alternatively, some, or all, of the cache for the processor set may be located “off chip.” In some computing environments, processor set **110** may be designed for working with qubits and performing quantum computing.

[0033] Computer readable program instructions are typically loaded onto computer **101** to cause a series of operational steps to be performed by processor set **110** of computer **101** and thereby effect a computer-implemented method, such that the instructions thus executed will instantiate the methods specified in flowcharts and/or narrative descriptions of computer-implemented methods included in this document (collectively referred to as “the inventive methods”). These computer readable program instructions are stored in various types of computer readable storage media, such as cache **121** and the other storage media discussed below. The program instructions, and associated data, are accessed by processor set **110** to control and direct performance of the inventive methods. In computing environment **100**, at least some of the instructions for performing the inventive methods may be stored in block **200** in persistent storage **113**.

[0034] COMMUNICATION FABRIC **111** is the signal conduction path that allows the various components of computer **101** to communicate with each other. Typically, this fabric is made of switches and electrically conductive paths, such as the switches and electrically conductive paths that make up buses, bridges, physical input/output ports and the like. Other types of signal communication paths may be used, such as fiber optic communication paths and/or wireless communication paths.

[0035] VOLATILE MEMORY **112** is any type of volatile memory now known or to be developed in the future. Examples include dynamic type random access memory (RAM) or static type RAM. Typically, volatile memory **112** is characterized by random access, but this is not required unless affirmatively indicated. In computer **101**, the volatile memory **112** is located in a single package and is internal to computer **101**, but, alternatively or additionally, the volatile memory may be distributed over multiple packages and/or located externally with respect to computer **101**.

[0036] PERSISTENT STORAGE **113** is any form of non-volatile storage for computers that is now known or to be developed in the future. The non-volatility of this storage means that the stored data is maintained regardless of whether power is being supplied to computer **101** and/or directly to persistent storage **113**. Persistent storage **113** may be a read only memory (ROM), but typically at least a portion of the persistent storage allows writing of data, deletion of data and re-writing of data. Some familiar forms of persistent storage include magnetic disks and solid state storage devices. Operating system **122** may take several forms, such as various known proprietary operating systems or open source Portable Operating System Interface-type operating systems that employ a kernel. The code included in block **200** typically includes at least some of the computer code involved in performing the inventive methods.

[0037] PERIPHERAL DEVICE SET **114** includes the set of peripheral devices of computer **101**. Data communication connections between the peripheral devices and the other components of computer **101** may be implemented in various ways, such as Bluetooth connections, Near-Field Communication (NFC) connections, connections made by cables (such as universal serial bus (USB) type cables), insertion-type connections (for example, secure digital (SD) card), connections made through local area communication networks and even connections made through wide area networks such as the internet. In various embodiments, UI device set **123** may include components such as a display screen, speaker, microphone, wearable devices (such as goggles and smart watches), keyboard, mouse, printer, touchpad, game controllers, and haptic devices. Storage **124** is external storage, such as an external hard drive, or insertable storage, such as an SD card. Storage **124** may be persistent and/or volatile. In some embodiments, storage **124** may take the form of a quantum computing storage device for storing data in the form of qubits. In embodiments where computer **101** is required to have a large amount of storage (for example, where computer **101** locally stores and manages a large database) then this storage may be provided by peripheral storage devices designed for storing very large amounts of data, such as a storage area network (SAN) that is shared by multiple, geographically distributed computers. IoT sensor set **125** is made up of sensors that can be used in Internet of Things applications. For example, one sensor may be a thermometer and another sensor may be a motion detector.

[0038] NETWORK MODULE **115** is the collection of computer software, hardware, and firmware that allows computer **101** to communicate with other computers through WAN **102**. Network module **115** may include hardware, such as modems or Wi-Fi signal transceivers, software for packetizing and/or de-packetizing data for communication network transmission, and/or web browser software for communicating data over the internet. In some embodiments, network control functions and network forwarding functions of network module **115** are performed on the same physical hardware device. In other embodiments (for example, embodiments that utilize software-defined networking (SDN)), the control functions and the forwarding functions of network module **115** are performed on physically separate devices, such that the control functions manage several different network hardware devices. Computer readable program instructions for performing the inventive methods can typically be downloaded to computer **101** from

an external computer or external storage device through a network adapter card or network interface included in network module 115.

[0039] WAN 102 is any wide area network (for example, the internet) capable of communicating computer data over non-local distances by any technology for communicating computer data, now known or to be developed in the future. In some embodiments, the WAN 102 may be replaced and/or supplemented by local area networks (LANs) designed to communicate data between devices located in a local area, such as a Wi-Fi network. The WAN and/or LANs typically include computer hardware such as copper transmission cables, optical transmission fibers, wireless transmission, routers, firewalls, switches, gateway computers and edge servers.

[0040] END USER DEVICE (EUD) 103 is any computer system that is used and controlled by an end user (for example, a customer of an enterprise that operates computer 101), and may take any of the forms discussed above in connection with computer 101. EUD 103 typically receives helpful and useful data from the operations of computer 101. For example, in a hypothetical case where computer 101 is designed to provide a recommendation to an end user, this recommendation would typically be communicated from network module 115 of computer 101 through WAN 102 to EUD 103. In this way, EUD 103 can display, or otherwise present, the recommendation to an end user. In some embodiments, EUD 103 may be a client device, such as thin client, heavy client, mainframe computer, desktop computer and so on.

[0041] REMOTE SERVER 104 is any computer system that serves at least some data and/or functionality to computer 101. Remote server 104 may be controlled and used by the same entity that operates computer 101. Remote server 104 represents the machine(s) that collect and store helpful and useful data for use by other computers, such as computer 101. For example, in a hypothetical case where computer 101 is designed and programmed to provide a recommendation based on historical data, then this historical data may be provided to computer 101 from remote database 130 of remote server 104.

[0042] PUBLIC CLOUD 105 is any computer system available for use by multiple entities that provides on-demand availability of computer system resources and/or other computer capabilities, especially data storage (cloud storage) and computing power, without direct active management by the user. Cloud computing typically leverages sharing of resources to achieve coherence and economies of scale. The direct and active management of the computing resources of public cloud 105 is performed by the computer hardware and/or software of cloud orchestration module 141. The computing resources provided by public cloud 105 are typically implemented by virtual computing environments that run on various computers making up the computers of host physical machine set 142, which is the universe of physical computers in and/or available to public cloud 105. The virtual computing environments (VCEs) typically take the form of virtual machines from virtual machine set 143 and/or containers from container set 144. It is understood that these VCEs may be stored as images and may be transferred among and between the various physical machine hosts, either as images or after instantiation of the VCE. Cloud orchestration module 141 manages the transfer and storage of images, deploys new instantiations of VCEs

and manages active instantiations of VCE deployments. Gateway 140 is the collection of computer software, hardware, and firmware that allows public cloud 105 to communicate through WAN 102.

[0043] Some further explanation of virtualized computing environments (VCEs) will now be provided. VCEs can be stored as “images.” A new active instance of the VCE can be instantiated from the image. Two familiar types of VCEs are virtual machines and containers. A container is a VCE that uses operating-system-level virtualization. This refers to an operating system feature in which the kernel allows the existence of multiple isolated user-space instances, called containers. These isolated user-space instances typically behave as real computers from the point of view of programs running in them. A computer program running on an ordinary operating system can utilize all resources of that computer, such as connected devices, files and folders, network shares, CPU power, and quantifiable hardware capabilities. However, programs running inside a container can only use the contents of the container and devices assigned to the container, a feature which is known as containerization.

[0044] PRIVATE CLOUD 106 is similar to public cloud 105, except that the computing resources are only available for use by a single enterprise. While private cloud 106 is depicted as being in communication with WAN 102, in other embodiments a private cloud may be disconnected from the internet entirely and only accessible through a local/private network. A hybrid cloud is a composition of multiple clouds of different types (for example, private, community or public cloud types), often respectively implemented by different vendors. Each of the multiple clouds remains a separate and discrete entity, but the larger hybrid cloud architecture is bound together by standardized or proprietary technology that enables orchestration, management, and/or data/application portability between the multiple constituent clouds. In this embodiment, public cloud 105 and private cloud 106 are both part of a larger hybrid cloud.

[0045] CLOUD COMPUTING SERVICES AND/OR MICROSERVICES (not separately shown in FIG. 1): private and public clouds 106 are programmed and configured to deliver cloud computing services and/or microservices (unless otherwise indicated, the word “microservices” shall be interpreted as inclusive of larger “services” regardless of size). Cloud services are infrastructure, platforms, or software that are typically hosted by third-party providers and made available to users through the internet. Cloud services facilitate the flow of user data from front-end clients (for example, user-side servers, tablets, desktops, laptops), through the internet, to the provider’s systems, and back. In some embodiments, cloud services may be configured and orchestrated according to as “as a service” technology paradigm where something is being presented to an internal or external customer in the form of a cloud computing service. As-a-Service offerings typically provide endpoints with which various customers interface. These endpoints are typically based on a set of APIs. One category of as-a-service offering is Platform as a Service (PaaS), where a service provider provisions, instantiates, runs, and manages a modular bundle of code that customers can use to instantiate a computing platform and one or more applications, without the complexity of building and maintaining the infrastructure typically associated with these things. Another category is Software as a Service (SaaS) where software is

centrally hosted and allocated on a subscription basis. SaaS is also known as on-demand software, web-based software, or web-hosted software. Four technological sub-fields involved in cloud services are: deployment, integration, on demand, and virtual private networks.

[0046] FIG. 2 depicts a diagram of a weighted counterfactual explanation in an environment in accordance with an illustrative embodiment. In a particular embodiment, the diagram 220 shows aspects of the application 200 of FIG. 1.

[0047] In the illustrated embodiment, the user 250 is shown interacting with an artificial intelligence (AI) system 280 comprising a machine learning (ML) model 230 and a weighted Counterfactual Engine 240. The artificial intelligence system detects a data instance for example a characteristic of the model e.g. a feature and a weight 260. The Counterfactual Engine 240 interacts 290 with the machine learning (ML) model 230 to compute a counterfactual objective and outputs a counterfactual explanation to the user 270.

[0048] In embodiments, the Counterfactual Engine may be a computational system including a large collection of simple neural units or processing elements (PEs) interconnected together. This may comprise processor nodes and connections between them, each node and connection performing computations, wherein the behavior of the nodes and connections is described by parameter values which either may be fixed or may be modifiable according to a specified learning or update rule. The local processing circuitry may include static random-access memory (SRAM), Direct memory access (DMA) and a central processing unit. In various embodiments, the CPU may include various types of computational processing units such as graphics processing units (GPUs), microcontrollers, or similar.

[0049] FIG. 3 depicts a diagram of a diverse counterfactual explanation in accordance with an illustrative embodiment. In a particular embodiment, the components of the diagram 300 shows aspects of the application 200 of FIG. 1.

[0050] In the illustrated embodiment, the user 360 is shown interacting with an artificial intelligence (AI) system 310 comprising a machine learning (ML) model 320 and a diverse Counterfactual Engine 330. The artificial intelligence system detects a data instance for example a characteristic of the model e.g. a feature and an iteration parameter k 340. The Counterfactual Engine 240 interacts 370 with the machine learning (ML) model 320 to compute counterfactual objectives over k iterations and outputs k counterfactual explanations to the user 350.

[0051] In embodiments, consider the setting in which there is a preference over which features should be perturbed when searching for counterfactuals. In embodiments, a model $f: \mathbb{R}^d \rightarrow \Delta^Y$ outputs a vector in the simplex of probabilities over $|Y|$ from an input $X \in \mathbb{X}$, where $\mathbb{X} \subseteq \mathbb{R}^d$. Given an instance $x_0 \in \mathbb{X}$, and a distance function $d: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^+$, the counterfactual objective is to find the closest sample to instance x_0 that changes the outcome of the model. A vector of non-negative feature weights (or costs) $w \in \mathbb{R}_+^d$ with larger values on features that, preferably, should remain invariant. Large weights discourage large perturbations i.e. the weight penalizes the perturbation of the feature. The cost, or weight vector w can be obtained by a cost preference. The regularization term with $R(\delta w)$, where $\delta w = w \odot \delta$ is modified denoting the element-wise product between each feature weight and its corresponding perturbation. $R(\cdot)$ for

example, may be a convex regularization function with a minimum in $\rightarrow 0$. The lambda parameter is a penalty coefficient that is optimized to ensure that the counterfactual condition is satisfied on the generated counterfactual. In some embodiments, the regularization function may comprise of distance functions including but not limited to Minkowski Distance, Manhattan Distance, and Euclidean Distance. The weighted counterfactual perturbation δ_{wcf} which is the closest sample to instance x_0 that changes the outcome of the machine learning model based on the weight/cost vector w is:

$$\delta_{wcf} = \underset{\delta: \delta + x_0 \in \mathbb{X}}{\operatorname{argmin}} wCF(\delta; w)$$

where $wCF(\delta; w)$ is the weighted counterfactual objective as described below

$$wCF(\delta; w) = \max_{\lambda} R(w \odot \delta) + \lambda \left[f_{y_0}(x_0 + \delta) - \max_{i \in Y - y_0} f_i(x_0 - \delta) \right]^+ \\ \text{with} \\ y^0 = \operatorname{argmax}_{i \in Y} f_i(x_0)$$

The weighted counterfactual explanation is $x_{cf} = x_0 + \delta_{wcf}$.

[0052] FIG. 4 depicts a flowchart diagram of a weighted counterfactual explanation in accordance with an illustrative embodiment. In a particular embodiment, the components 400 are representative of aspects of the application 200 of FIG. 1.

[0053] In the illustrated embodiment, the flow chart starts with a detection by an artificial intelligence system of a feature of a machine learning model 410. Responsive to detecting the feature, computing by a Counterfactual Engine a counterfactual objective comprising modifying a product of a weight and a perturbation of the feature 420 that penalizes each feature perturbation according to the weighting vector w , and minimizing the modified product 430. Next, transforming the counterfactual objective into a counterfactual of the machine learning model 440.

[0054] In some embodiments, multiple diverse counterfactuals are generated by leveraging the weighting vector δ_{wcf} . For example, initialize $w = 1^d$ and counterfactual set $\mathcal{C} = \emptyset$. Iterative for k iterations, compute a weighted counterfactual by minimizing:

[0055] $\delta_{cf,t} = \operatorname{argmin} wCF(\delta; w_t)$ where wCF is the weighted counterfactual objective function described above.

[0056] Add the counterfactual to $\mathcal{C}$, update weight vector w to prefer features in unexplored directions. In embodiments, the current weight is updated as described by the function:

$$w_{t+1} = \left[\Delta d - [w_t + \gamma |\delta_{cf,t}|, t] \right]$$

Here $|\delta_{cf,t}|$ indicates the element-wise absolute value of the perturbations of the t-th counterfactual. $\gamma > 1$ is a factor that

scales the aggregation of the previously perturbed features, and $\Pi_{\Delta d-1}[\cdot]$ denotes the Euclidean projection operator into the simplex $\Delta d-1$.

[0057] In embodiments, the iterations recover cost-aware counterfactuals to update a set of counterfactuals, and use the sequence of previous counterfactuals to compute the current weighting vector w . This promotes the generation of counterfactuals that do not modify features that were previously perturbed unless it is necessary. Note that, as the iterations increases, features that have not been perturbed see their corresponding weight decay to zero, which encourages these features to be perturbed for the subsequent counterfactual perturbation.

[0058] FIG. 5 depicts a flowchart diagram of a diverse counterfactual explanation in accordance with an illustrative embodiment. In a particular embodiment, the system components 500 are representative of aspects of the application 200 of FIG. 1.

[0059] In the illustrated embodiment, the flow chart starts with a detection by an artificial intelligence system of a feature of a machine learning model 510. Responsive to detecting the feature, computing by a Counterfactual Engine a counterfactual objective comprising modifying a product of a weight and a perturbation of the feature 520 that penalizes each feature perturbation according to the weighting vector w , and minimizing the modified product 530. Next, transforming the counterfactual objective into a counterfactual of the machine learning model 540. The steps 520, 530, and 540 i.e. computing and transforming are iterated k iterations and the weight adjusted 550. The iterating updates a set of counterfactuals.

[0060] FIG. 6 depicts a system diagram in accordance with an illustrative embodiment. In a particular embodiment, the system components 600 are representative of aspects of the application 200 of FIG. 1.

[0061] In the illustrated embodiment, the Detection Component 630 of an artificial intelligence system detects a feature of a machine learning model. A Counterfactual Engine Component 660 interacts with the Machine Learning Component 650 responsive to the detecting the feature, to compute a counterfactual objective comprising modifying a product of a weight and a perturbation of the feature and minimizing the modified product. Next, transforming the counterfactual objective into a counterfactual of the machine learning model. A central processing unit (CPU) 670 performs operations on the various components. In embodiments, the operations of computing by the Counterfactual Engine Component comprising modifying and then minimizing the product may involve accelerating processing by optimizing GPU/CPU interaction including processor nodes and connections.

[0062] The embodiments described herein may provide counterfactual explanations for a loan application system. For example, if the loan application system rejects the user, the user ask what changes to the income feature of the loan application model will change the result. The system may designate a weight to the income feature, and the Counterfactual Engine modifies the product of the feature and weight, and minimizes the product to generate a counterfactual objective. The counterfactual objective and the starting income value are transformed into a counterfactual explanation. In another embodiment, the Counterfactual

Engine iterates through k products of weights and features to update a set of counterfactual explanations. This set may include unexplored features.

[0063] The following definitions and abbreviations are to be used for the interpretation of the claims and the specification. As used herein, the terms “comprises,” “comprising,” “includes,” “including,” “has,” “having,” “contains” or “containing,” or any other variation thereof, are intended to cover a non-exclusive inclusion. For example, a composition, a mixture, process, method, article, or apparatus that comprises a list of elements is not necessarily limited to only those elements but can include other elements not expressly listed or inherent to such composition, mixture, process, method, article, or apparatus.

[0064] Additionally, the term “illustrative” is used herein to mean “serving as an example, instance or illustration.” Any embodiment or design described herein as “illustrative” is not necessarily to be construed as preferred or advantageous over other embodiments or designs. The terms “at least one” and “one or more” are understood to include any integer number greater than or equal to one, i.e., one, two, three, four, etc. The terms “a plurality” are understood to include any integer number greater than or equal to two, i.e., two, three, four, five, etc. The term “connection” can include an indirect “connection” and a direct “connection.”

[0065] References in the specification to “one embodiment,” “an embodiment,” “an example embodiment,” etc., indicate that the embodiment described can include a particular feature, structure, or characteristic, but every embodiment may or may not include the particular feature, structure, or characteristic. Moreover, such phrases are not necessarily referring to the same embodiment. Further, when a particular feature, structure, or characteristic is described in connection with an embodiment, it is submitted that it is within the knowledge of one skilled in the art to affect such feature, structure, or characteristic in connection with other embodiments whether or not explicitly described.

[0066] The terms “about,” “substantially,” “approximately,” and variations thereof, are intended to include the degree of error associated with measurement of the particular quantity based upon the equipment available at the time of filing the application. For example, “about” can include a range of $\pm 8\%$ or 5% , or 2% of a given value.

[0067] The descriptions of the various embodiments of the present invention have been presented for purposes of illustration but are not intended to be exhaustive or limited to the embodiments disclosed. Many modifications and variations will be apparent to those of ordinary skill in the art without departing from the scope and spirit of the described embodiments. The terminology used herein was chosen to best explain the principles of the embodiments, the practical application or technical improvement over technologies found in the marketplace, or to enable others of ordinary skill in the art to understand the embodiments described herein.

[0068] The descriptions of the various embodiments of the present invention have been presented for purposes of illustration but are not intended to be exhaustive or limited to the embodiments disclosed. Many modifications and variations will be apparent to those of ordinary skill in the art without departing from the scope and spirit of the described embodiments. The terminology used herein was chosen to best explain the principles of the embodiments, the practical application or technical improvement over tech-

nologies found in the marketplace, or to enable others of ordinary skill in the art to understand the embodiments described herein.

[0069] Thus, a computer implemented method, system or apparatus, and computer program product are provided in the illustrative embodiments for managing participation in online communities and other related features, functions, or operations. Where an embodiment or a portion thereof is described with respect to a type of device, the computer implemented method, system or apparatus, the computer program product, or a portion thereof, are adapted or configured for use with a suitable and comparable manifestation of that type of device.

[0070] Where an embodiment is described as implemented in an application, the delivery of the application in a Software as a Service (SaaS) model is contemplated within the scope of the illustrative embodiments. In a SaaS model, the capability of the application implementing an embodiment is provided to a user by executing the application in a cloud infrastructure. The user can access the application using a variety of client devices through a thin client interface such as a web browser (e.g., web-based e-mail), or other light-weight client-applications. The user does not manage or control the underlying cloud infrastructure including the network, servers, operating systems, or the storage of the cloud infrastructure. In some cases, the user may not even manage or control the capabilities of the SaaS application. In some other cases, the SaaS implementation of the application may permit a possible exception of limited user-specific application configuration settings.

[0071] Embodiments of the present invention may also be delivered as part of a service engagement with a client corporation, nonprofit organization, government entity, internal organizational structure, or the like. Aspects of these embodiments may include configuring a computer system to perform, and deploying software, hardware, and web services that implement, some or all of the methods described herein. Aspects of these embodiments may also include analyzing the client's operations, creating recommendations responsive to the analysis, building systems that implement portions of the recommendations, integrating the systems into existing processes and infrastructure, metering use of the systems, allocating expenses to users of the systems, and billing for use of the systems. Although the above embodiments of present invention each have been described by stating their individual advantages, respectively, present invention is not limited to a particular combination thereof. To the contrary, such embodiments may also be combined in any way and number according to the intended deployment of present invention without losing their beneficial effects.

What is claimed is:

1. A computer-implemented method comprising:
detecting by an artificial intelligence system a feature of a machine learning model;
responsive to detecting the feature of the machine learning model, computing by a Counterfactual Engine of the artificial intelligence system a counterfactual objective based on modifying a product of a weight and a perturbation of the feature; and
transforming by the Counterfactual Engine a counterfactual based on the counterfactual objective.
2. The computer-implemented method of claim 1, wherein the weight penalizes the perturbation of the feature.

3. The computer-implemented method of claim 1, wherein the weight is a maximum value for the feature that is invariant.

4. The computer-implemented method of claim 1, wherein the modifying further comprises minimizing the product based on a distance algorithm.

5. The computer-implemented method of claim 1, wherein the weight comprises a cost preference.

6. The computer-implemented method of claim 1, further comprising iterating the computing and the transforming wherein the iterating updates a set of counterfactuals and a current weighting vector.

7. The computer-implemented method of claim 6, wherein the current weighting vector is computed based on a previous counterfactual from the set of counterfactuals.

8. A computer program product comprising one or more computer readable storage media, and program instructions collectively stored on the one or more computer readable storage media, the program instructions executable by a processor to cause the processor to perform operations comprising:

detecting by an artificial intelligence system a feature of a machine learning model;

responsive to detecting the feature of the machine learning model, computing by a Counterfactual Engine of the artificial intelligence system a counterfactual objective based on modifying a product of a weight and a perturbation of the feature; and

transforming by the Counterfactual Engine a counterfactual based on the counterfactual objective.

9. The computer program product of claim 8, wherein the weight penalizes the perturbation of the feature.

10. The computer program product of claim 8, wherein the weight is a maximum value for the feature that is invariant.

11. The computer program product of claim 8, wherein the modifying further comprises minimizing the product based on a distance algorithm.

12. The computer program product of claim 8, wherein the weight comprises a cost preference.

13. The computer program product of claim 8, further comprising iterating the computing and the transforming wherein the iterating updates a set of counterfactuals and a current weighting vector.

14. The computer program product of claim 13, wherein the current weighting vector is computed based on a previous counterfactual from the set of counterfactuals.

15. A computer system comprising a processor and one or more computer readable storage media, and program instructions collectively stored on the one or more computer readable storage media, the program instructions executable by the processor to cause the processor to perform operations comprising:

detecting by an artificial intelligence system a feature of a machine learning model;

responsive to detecting the feature of the machine learning model, computing by a Counterfactual Engine of the artificial intelligence system a counterfactual objective based on modifying a product of a weight and a perturbation of the feature; and

transforming by the Counterfactual Engine a counterfactual based on the counterfactual objective.

16. The computer system of claim 15, wherein the weight penalizes the perturbation of the feature.

17. The computer system of claim **15**, wherein the modifying further comprises minimizing the product based on a distance algorithm.

18. The computer system of claim **15**, wherein the weight comprises a cost preference.

19. The computer system of claim **15**, further comprising iterating the computing and the transforming wherein the iterating updates a set of counterfactuals and a current weighting vector.

20. The computer system of claim **19**, wherein the current weighting vector is computed based on a previous counterfactual from the set of counterfactuals.

* * * * *